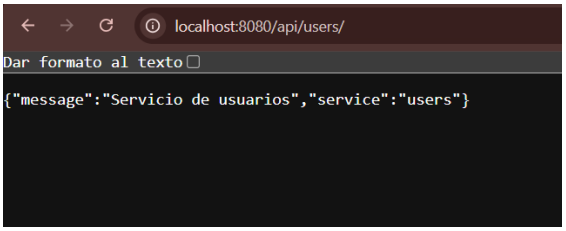
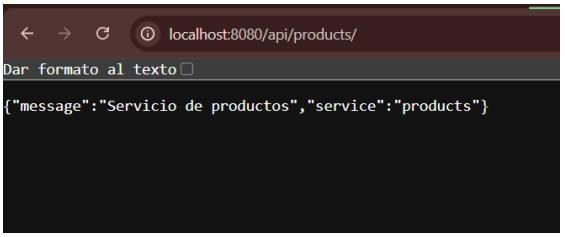
Workshop 2 - 03docker

```
feijo@Valenfei MINGW64 ~/03-docker/WS2/taller2-reverse-proxy (master)
$ cat > users/app.py << 'EOF'
from flask import Flask, jsonify

app = Flask(__name__)

@app.get("/")
def home():
    return jsonify({
        "service": "users",
        "message": "Servicio de usuarios"
    })

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
EOF

feijo@Valenfei MINGW64 ~/03-docker/WS2/taller2-reverse-proxy (master)
$ cat > users/requirements.txt << 'EOF'
Flask
EOF

feijo@Valenfei MINGW64 ~/03-docker/WS2/taller2-reverse-proxy (master)
$ cat > users/Dockerfile << 'EOF'
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 5000

CMD ["python", "app.py"]
EOF

feijo@Valenfei MINGW64 ~/03-docker/WS2/taller2-reverse-proxy (master)
$
```

```
feijo@Valenfei MINGW64 ~/03-docker/WS2/taller2-reverse-proxy (master)
$ cat > products/app.py << 'EOF'
from flask import Flask, jsonify

app = Flask(__name__)

@app.get("/")
def home():
    return jsonify({
        "service": "products",
        "message": "Servicio de productos"
    })

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
EOF

feijo@Valenfei MINGW64 ~/03-docker/WS2/taller2-reverse-proxy (master)
$ cat > products/requirements.txt << 'EOF'
Flask
EOF

feijo@Valenfei MINGW64 ~/03-docker/WS2/taller2-reverse-proxy (master)
$ cat > products/Dockerfile << 'EOF'
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 5000

CMD ["python", "app.py"]
EOF

feijo@Valenfei MINGW64 ~/03-docker/WS2/taller2-reverse-proxy (master)
$ cat > nginx/nginx.conf << 'EOF'
server {
    listen 80;

    location /api/users/ {
        proxy_pass http://users:5000/;
    }

    location /api/products/ {
        proxy_pass http://products:5000/;
    }
}
EOF
```

```
feijo@Valenfei MINGW64 ~/03-docker/WS2/taller2-reverse-proxy (master)
$ cat > docker-compose.yml << 'EOF'
services:
  users:
    build: ./users

  products:
    build: ./products

  nginx:
    image: nginx:alpine
    ports:
      - "8080:80"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      - users
      - products
EOF

feijo@Valenfei MINGW64 ~/03-docker/WS2/taller2-reverse-proxy (master)
$ docker compose up --build
 Image taller2-reverse-proxy-users Building
 Image taller2-reverse-proxy-products Building
```






```
feijo@Valenfei MINGW64 ~/03-docker/WS2/taller2-reverse-proxy (master)
$ docker compose down
 Container taller2-reverse-proxy-nginx-1 Stopping
 Container taller2-reverse-proxy-nginx-1 Stopped
 Container taller2-reverse-proxy-nginx-1 Removing
 Container taller2-reverse-proxy-nginx-1 Removed
 Container taller2-reverse-proxy-products-1 Stopping
 Container taller2-reverse-proxy-users-1 Stopping
 Container taller2-reverse-proxy-products-1 Stopped
 Container taller2-reverse-proxy-products-1 Removing
 Container taller2-reverse-proxy-products-1 Removed
 Container taller2-reverse-proxy-users-1 Stopped
 Container taller2-reverse-proxy-users-1 Removing
 Container taller2-reverse-proxy-users-1 Removed
 Network taller2-reverse-proxy_default Removing
 Network taller2-reverse-proxy_default Removed

feijo@Valenfei MINGW64 ~/03-docker/WS2/taller2-reverse-proxy (master)
$ cat > docker-compose.yml << 'EOF'
services:
  users1:
    build: ./users

  users2:
    build: ./users

  products:
    build: ./products

  nginx:
    image: nginx:alpine
    ports:
      - "8080:80"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      - users1
      - users2
      - products
EOF
```

```
feijo@Valenfei MINGW64 ~/03-docker/WS2/taller2-reverse-proxy (master)
$ cat > nginx/nginx.conf << 'EOF'
upstream users_backend {
    server users1:5000;
    server users2:5000;
}

server {
    listen 80;

    location /api/users/ {
        proxy_pass http://users_backend/;
    }

    location /api/products/ {
        proxy_pass http://products:5000/;
    }
}
EOF

feijo@Valenfei MINGW64 ~/03-docker/WS2/taller2-reverse-proxy (master)
$ docker compose up --build
 Image taller2-reverse-proxy-users1 Building
 Image taller2-reverse-proxy-users2 Building
 Image taller2-reverse-proxy-products Building
#1 [internal] load local bake definitions
#1 reading from stdin 1.66kB 0.0s done
```

```
feijo@Valenfei MINGW64 ~/03-docker/WS2/taller2-reverse-proxy (master)
$ for i in {1..12}; do curl -s http://localhost:8080/api/users/; echo; done
{"message":"Servicio de usuarios","service":"users"}
{"message":"Servicio de usuarios","service":"users"}
{"message":"Servicio de usuarios","service":"users"}
{"message":"Servicio de usuarios","service":"users"}
{"message":"Servicio de usuarios","service":"users"}
{"message":"Servicio de usuarios","service":"users"}
{"message":"Servicio de usuarios","service":"users"}
{"message":"Servicio de usuarios","service":"users"}
{"message":"Servicio de usuarios","service":"users"}
{"message":"Servicio de usuarios","service":"users"}
{"message":"Servicio de usuarios","service":"users"}
{"message":"Servicio de usuarios","service":"users"}
```

- Hace falta el Reto

Diferencia entre Routing puro y Load Balancing

Routing puro (Parte 3):

Nginx recibe la solicitud y decide **qué servicio** debe atenderla, según la ruta (`/api/users/` → `users`, `/api/products/` → `products`). Cada ruta tiene un único backend fijo. No hay elección entre varias opciones equivalentes — es una decisión de "a dónde" enviar, no de "cuál instancia entre varias".

Load Balancing (Parte 5):

Nginx recibe la solicitud para una misma ruta (`/api/users/`) y decide **cuál instancia** entre varias equivalentes la atiende (`users1` o `users2`), usando una estrategia de distribución (Round Robin). Aquí sí hay múltiples backends intercambiables detrás de un mismo `upstream`.

En una frase:

Routing = distribuir por *tipo de servicio*. Load balancing = distribuir por *instancia* dentro del mismo servicio.